

ann

April 11, 2025

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report, roc_auc_score
from collections import Counter
from imblearn.over_sampling import SMOTE
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam
from collections import Counter
```

```
2025-04-11 16:57:52.276698: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32]
Could not find cuda drivers on your machine, GPU will not be used.
2025-04-11 16:57:52.570013: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:32]
Could not find cuda drivers on your machine, GPU will not be used.
2025-04-11 16:57:52.665427: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1744390672.988013    15103 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1744390673.053079    15103 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
W0000 00:00:1744390673.692739    15103 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1744390673.692776    15103 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1744390673.692780    15103 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
```

target more than once.

W0000 00:00:1744390673.692783 15103 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

2025-04-11 16:57:53.745384: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

```
[2]: # df_original = pd.read_csv('Children Recode_final.csv')
# df_original['Malnurished'] = df_original[['Underweight', 'Stunting', 'Wasting']].max(axis=1)
# df = df_original.drop(['Underweight', 'Stunting', 'Wasting'], axis = 1)
# df.head()

# X = df.drop(columns=['Malnurished'])
# y = df['Malnurished']

# # Train-test Split
# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state= 12)

# # Columns to scale
# columns_to_scale = ['Child_age', 'Age_first_sex', 'BMI', 'Mother_age_current', 'Mother_age_at_first_birth']
# scaler = StandardScaler()

# # Make copies of training and test sets
# X_train_scaled = X_train.copy()
# X_test_scaled = X_test.copy()

# # Scale only selected columns
# X_train_scaled[columns_to_scale] = scaler.fit_transform(X_train[columns_to_scale])
# X_test_scaled[columns_to_scale] = scaler.transform(X_test[columns_to_scale])

# # Apply SMOTE
# sm = SMOTE(random_state=42)
# X_train_sm, y_train_sm = sm.fit_resample(X_train_scaled, y_train)

# print("Before SMOTE:", Counter(y_train))
# print("After SMOTE:", Counter(y_train_sm))
```

ANN Model

```
[3]: ## Build the ANN model
# model = Sequential([
#     Dense(64, input_dim=X_train_sm.shape[1], activation='relu'),
#     Dropout(0.3),
#     Dense(32, activation='relu'),
#     Dense(1, activation='sigmoid') # Binary output
# ])

## Compile the model
# model.compile(
#     optimizer=Adam(learning_rate=0.001),
#     loss='binary_crossentropy',
#     metrics=['Recall', 'Precision']
# )

## Train the model
# history = model.fit(
#     X_train_sm, y_train_sm,
#     epochs=50,
#     batch_size=32,
#     validation_split=0.2,
#     verbose=1
# )
```

```
[4]: ## Evaluate on test set
# y_pred_prob = model.predict(X_test_scaled).flatten()
# y_pred = (y_pred_prob >= 0.5).astype(int)

# print("Classification Report on Test Set:")
# print(classification_report(y_test, y_pred))
```

```
[5]: # Load Data
df_original = pd.read_csv('df.csv')
df_original['Malnurished'] = df_original[['Underweight', 'Stunting',
↳ 'Wasting']].max(axis=1)
df = df_original.drop(['Underweight', 'Stunting', 'Wasting'], axis = 1)

# Train-test Split
X = df.drop(columns=['Malnurished'])
y = df['Malnurished']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20,
↳ stratify = y, random_state = 42)

# Standard Scaler
columns_to_scale = ['Child_age', 'BMI', 'Mother_age_current']
scaler = StandardScaler()
X_train[columns_to_scale] = scaler.fit_transform(X_train[columns_to_scale])
```

```
X_test[columns_to_scale] = scaler.transform(X_test[columns_to_scale])
```

```
[6]: smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)
```

```
[7]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
from sklearn.utils import class_weight
```

```
[8]: # Step 5: Build the Artificial Neural Network model
model = Sequential()
# Input layer with 32 neurons; adjust the number of neurons based on your
↳ dataset's complexity
model.add(Dense(32, activation='relu', input_shape=(X_train.shape[1],)))
# You can add a dropout layer to reduce overfitting
model.add(Dropout(0.2))
# A hidden layer with 16 neurons
model.add(Dense(16, activation='relu'))
# Output layer with a single neuron and sigmoid activation for binary
↳ classification
model.add(Dense(1, activation='sigmoid'))

# Step 6: Compile the model
# Use binary_crossentropy as the loss function for binary classification
model.compile(optimizer=Adam(learning_rate=0.001), loss='binary_crossentropy',
↳ metrics=['recall'])

# Step 7: Train the model
# Use early stopping to halt training once the validation loss stops decreasing
early_stop = EarlyStopping(monitor='val_loss', patience=10,
↳ restore_best_weights=True)

history = model.fit(
    X_train,
    y_train,
    validation_split=0.2, # Use 20% of the training set for validation
    epochs=100,          # You can increase epochs if your dataset is larger
    batch_size=16,
    callbacks=[early_stop],
    verbose=1
```

```

)

# Step 8: Evaluate the model on the test set
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)

# Make predictions on the test set
y_pred = model.predict(X_test).ravel()
# Convert probabilities to binary outcomes (using 0.5 as the threshold)
y_pred_binary = (y_pred > 0.5).astype(int)

# Display evaluation metrics
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_binary))
print("Classification Report:")
print(classification_report(y_test, y_pred_binary))

```

```

/usr/local/python/3.12.1/lib/python3.12/site-
packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2025-04-11 16:57:58.205466: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)

```

```

Epoch 1/100
117/117          1s 3ms/step -
loss: 0.6711 - recall: 0.0232 - val_loss: 0.8671 - val_recall: 0.0171
Epoch 2/100
117/117          0s 2ms/step -
loss: 0.6477 - recall: 0.0449 - val_loss: 0.8284 - val_recall: 0.1389
Epoch 3/100
117/117          0s 2ms/step -
loss: 0.6475 - recall: 0.0750 - val_loss: 0.8548 - val_recall: 0.1175
Epoch 4/100
117/117          0s 2ms/step -
loss: 0.6367 - recall: 0.0791 - val_loss: 0.8541 - val_recall: 0.1474
Epoch 5/100
117/117          0s 2ms/step -
loss: 0.6493 - recall: 0.1723 - val_loss: 0.8761 - val_recall: 0.1154
Epoch 6/100
117/117          0s 3ms/step -
loss: 0.6315 - recall: 0.1261 - val_loss: 0.8141 - val_recall: 0.3141
Epoch 7/100
117/117          0s 2ms/step -
loss: 0.6202 - recall: 0.1956 - val_loss: 0.8022 - val_recall: 0.3376
Epoch 8/100

```

```

117/117          0s 2ms/step -
loss: 0.6190 - recall: 0.1706 - val_loss: 0.8445 - val_recall: 0.2842
Epoch 9/100
117/117          1s 2ms/step -
loss: 0.6144 - recall: 0.1859 - val_loss: 0.7836 - val_recall: 0.4103
Epoch 10/100
117/117          0s 2ms/step -
loss: 0.6044 - recall: 0.2210 - val_loss: 0.7962 - val_recall: 0.3932
Epoch 11/100
117/117          0s 2ms/step -
loss: 0.6108 - recall: 0.2704 - val_loss: 0.7619 - val_recall: 0.4487
Epoch 12/100
117/117          0s 2ms/step -
loss: 0.6053 - recall: 0.2885 - val_loss: 0.8150 - val_recall: 0.3846
Epoch 13/100
117/117          0s 2ms/step -
loss: 0.5927 - recall: 0.2956 - val_loss: 0.7652 - val_recall: 0.4658
Epoch 14/100
117/117          0s 2ms/step -
loss: 0.5990 - recall: 0.3098 - val_loss: 0.7482 - val_recall: 0.4829
Epoch 15/100
117/117          0s 2ms/step -
loss: 0.6047 - recall: 0.3555 - val_loss: 0.7320 - val_recall: 0.5299
Epoch 16/100
117/117          0s 2ms/step -
loss: 0.5891 - recall: 0.3126 - val_loss: 0.6981 - val_recall: 0.5897
Epoch 17/100
117/117          0s 2ms/step -
loss: 0.6025 - recall: 0.3932 - val_loss: 0.7170 - val_recall: 0.5769
Epoch 18/100
117/117          0s 2ms/step -
loss: 0.5856 - recall: 0.3402 - val_loss: 0.6852 - val_recall: 0.6132
Epoch 19/100
117/117          0s 2ms/step -
loss: 0.5839 - recall: 0.4250 - val_loss: 0.7182 - val_recall: 0.5577
Epoch 20/100
117/117          0s 2ms/step -
loss: 0.5941 - recall: 0.3258 - val_loss: 0.7335 - val_recall: 0.5427
Epoch 21/100
117/117          0s 2ms/step -
loss: 0.5884 - recall: 0.3888 - val_loss: 0.7224 - val_recall: 0.5620
Epoch 22/100
117/117          0s 2ms/step -
loss: 0.5694 - recall: 0.3924 - val_loss: 0.7146 - val_recall: 0.5812
Epoch 23/100
117/117          0s 2ms/step -
loss: 0.5736 - recall: 0.3998 - val_loss: 0.6906 - val_recall: 0.5983
Epoch 24/100

```

117/117 0s 2ms/step -
loss: 0.5745 - recall: 0.4697 - val_loss: 0.7784 - val_recall: 0.4744
Epoch 25/100
117/117 0s 2ms/step -
loss: 0.5541 - recall: 0.4196 - val_loss: 0.6711 - val_recall: 0.6218
Epoch 26/100
117/117 0s 2ms/step -
loss: 0.5709 - recall: 0.4691 - val_loss: 0.6834 - val_recall: 0.6047
Epoch 27/100
117/117 0s 3ms/step -
loss: 0.5792 - recall: 0.4605 - val_loss: 0.6554 - val_recall: 0.6517
Epoch 28/100
117/117 0s 2ms/step -
loss: 0.5653 - recall: 0.4740 - val_loss: 0.6733 - val_recall: 0.6111
Epoch 29/100
117/117 0s 2ms/step -
loss: 0.5638 - recall: 0.4672 - val_loss: 0.6960 - val_recall: 0.5962
Epoch 30/100
117/117 0s 2ms/step -
loss: 0.5522 - recall: 0.4384 - val_loss: 0.6269 - val_recall: 0.6667
Epoch 31/100
117/117 0s 2ms/step -
loss: 0.5422 - recall: 0.4947 - val_loss: 0.6213 - val_recall: 0.6816
Epoch 32/100
117/117 0s 2ms/step -
loss: 0.5514 - recall: 0.4841 - val_loss: 0.6946 - val_recall: 0.5705
Epoch 33/100
117/117 0s 2ms/step -
loss: 0.5667 - recall: 0.4535 - val_loss: 0.6839 - val_recall: 0.5897
Epoch 34/100
117/117 0s 2ms/step -
loss: 0.5567 - recall: 0.4934 - val_loss: 0.6633 - val_recall: 0.6197
Epoch 35/100
117/117 0s 2ms/step -
loss: 0.5421 - recall: 0.5091 - val_loss: 0.6362 - val_recall: 0.6496
Epoch 36/100
117/117 0s 2ms/step -
loss: 0.5345 - recall: 0.4878 - val_loss: 0.6281 - val_recall: 0.6709
Epoch 37/100
117/117 0s 2ms/step -
loss: 0.5457 - recall: 0.4933 - val_loss: 0.6331 - val_recall: 0.6538
Epoch 38/100
117/117 0s 2ms/step -
loss: 0.5490 - recall: 0.5161 - val_loss: 0.6596 - val_recall: 0.6261
Epoch 39/100
117/117 0s 2ms/step -
loss: 0.5465 - recall: 0.5016 - val_loss: 0.6445 - val_recall: 0.6389
Epoch 40/100

```

117/117          0s 2ms/step -
loss: 0.5271 - recall: 0.5212 - val_loss: 0.6615 - val_recall: 0.6068
Epoch 41/100
117/117          0s 2ms/step -
loss: 0.5170 - recall: 0.4786 - val_loss: 0.5926 - val_recall: 0.7115
Epoch 42/100
117/117          0s 2ms/step -
loss: 0.5314 - recall: 0.5575 - val_loss: 0.6010 - val_recall: 0.6923
Epoch 43/100
117/117          0s 2ms/step -
loss: 0.5326 - recall: 0.5129 - val_loss: 0.6425 - val_recall: 0.6325
Epoch 44/100
117/117          0s 2ms/step -
loss: 0.5216 - recall: 0.5339 - val_loss: 0.6099 - val_recall: 0.6902
Epoch 45/100
117/117          0s 2ms/step -
loss: 0.5358 - recall: 0.5039 - val_loss: 0.6139 - val_recall: 0.6688
Epoch 46/100
117/117          0s 2ms/step -
loss: 0.5245 - recall: 0.5498 - val_loss: 0.6043 - val_recall: 0.6774
Epoch 47/100
117/117          0s 2ms/step -
loss: 0.5122 - recall: 0.5706 - val_loss: 0.5991 - val_recall: 0.6838
Epoch 48/100
117/117          0s 2ms/step -
loss: 0.5130 - recall: 0.5914 - val_loss: 0.5878 - val_recall: 0.7030
Epoch 49/100
117/117          0s 2ms/step -
loss: 0.5132 - recall: 0.5440 - val_loss: 0.6140 - val_recall: 0.6795
Epoch 50/100
117/117          0s 2ms/step -
loss: 0.5140 - recall: 0.5821 - val_loss: 0.5825 - val_recall: 0.7094
Epoch 51/100
117/117          0s 2ms/step -
loss: 0.5043 - recall: 0.5572 - val_loss: 0.6024 - val_recall: 0.6880
Epoch 52/100
117/117          0s 2ms/step -
loss: 0.5015 - recall: 0.5721 - val_loss: 0.6074 - val_recall: 0.6752
Epoch 53/100
117/117          0s 2ms/step -
loss: 0.4914 - recall: 0.5551 - val_loss: 0.6088 - val_recall: 0.6752
Epoch 54/100
117/117          0s 2ms/step -
loss: 0.4914 - recall: 0.5658 - val_loss: 0.5932 - val_recall: 0.6880
Epoch 55/100
117/117          0s 2ms/step -
loss: 0.5234 - recall: 0.5763 - val_loss: 0.5897 - val_recall: 0.6880
Epoch 56/100

```



```

117/117          0s 2ms/step -
loss: 0.5095 - recall: 0.5160 - val_loss: 0.6173 - val_recall: 0.6752
Epoch 57/100
117/117          0s 2ms/step -
loss: 0.4980 - recall: 0.5025 - val_loss: 0.5486 - val_recall: 0.7308
Epoch 58/100
117/117          0s 2ms/step -
loss: 0.4987 - recall: 0.5744 - val_loss: 0.5389 - val_recall: 0.7415
Epoch 59/100
117/117          0s 2ms/step -
loss: 0.5031 - recall: 0.6108 - val_loss: 0.5873 - val_recall: 0.6880
Epoch 60/100
117/117          0s 2ms/step -
loss: 0.5113 - recall: 0.5885 - val_loss: 0.6259 - val_recall: 0.6389
Epoch 61/100
117/117          0s 2ms/step -
loss: 0.4872 - recall: 0.5808 - val_loss: 0.6173 - val_recall: 0.6645
Epoch 62/100
117/117          0s 2ms/step -
loss: 0.4853 - recall: 0.5852 - val_loss: 0.5790 - val_recall: 0.7073
Epoch 63/100
117/117          0s 2ms/step -
loss: 0.4727 - recall: 0.6314 - val_loss: 0.5100 - val_recall: 0.7585
Epoch 64/100
117/117          0s 2ms/step -
loss: 0.4787 - recall: 0.6281 - val_loss: 0.5525 - val_recall: 0.7201
Epoch 65/100
117/117          0s 2ms/step -
loss: 0.4900 - recall: 0.5958 - val_loss: 0.5894 - val_recall: 0.6816
Epoch 66/100
117/117          0s 2ms/step -
loss: 0.4712 - recall: 0.5822 - val_loss: 0.5562 - val_recall: 0.7308
Epoch 67/100
117/117          0s 2ms/step -
loss: 0.4980 - recall: 0.5691 - val_loss: 0.5256 - val_recall: 0.7607
Epoch 68/100
117/117          0s 2ms/step -
loss: 0.4559 - recall: 0.6965 - val_loss: 0.5452 - val_recall: 0.7329
Epoch 69/100
117/117          0s 2ms/step -
loss: 0.4569 - recall: 0.6564 - val_loss: 0.5345 - val_recall: 0.7350
Epoch 70/100
117/117          0s 2ms/step -
loss: 0.4850 - recall: 0.6102 - val_loss: 0.5624 - val_recall: 0.7137
Epoch 71/100
117/117          0s 2ms/step -
loss: 0.4611 - recall: 0.5756 - val_loss: 0.5177 - val_recall: 0.7628
Epoch 72/100

```

```

117/117          0s 3ms/step -
loss: 0.4664 - recall: 0.6920 - val_loss: 0.6267 - val_recall: 0.6709
Epoch 73/100
117/117          0s 3ms/step -
loss: 0.4698 - recall: 0.6095 - val_loss: 0.5312 - val_recall: 0.7607
14/14           0s 2ms/step - loss:
0.7146 - recall: 0.4196
Test Accuracy: 0.4161073863506317
14/14           0s 3ms/step
Confusion Matrix:
[[212  80]
 [ 87  62]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.71	0.73	0.72	292
1	0.44	0.42	0.43	149
accuracy			0.62	441
macro avg	0.57	0.57	0.57	441
weighted avg	0.62	0.62	0.62	441

```

[9]: from sklearn.metrics import average_precision_score
print(f'Average Precision: {average_precision_score(y_test, y_pred)}')

```

Average Precision: 0.4193494702829542